

Enterprise Policy Interceptor Architecture for Agentic AI

1. *Journal of the American Medical Association*, 1997; 277: 1039-1043.

Every numbered decision point (P1–P7) is an independent Cedar policy evaluation. A positive decision at P1 does NOT grant permission at P3. Authorization is re-evaluated at every action boundary. This is the fundamental principle of Zero Trust for agents.

Step	Authorization Question	Principal	Policy Engine	Context Required
Agent Invocation	Can this user invoke agent type X?	User	Cedar AVP	capabilities, mfa, risk_score
Tool Selection	Is tool T in this agent's permitted toolset?	Agent	Cedar AVP	agent_type, user_capabilities

Step	Authorization Question	Principal	Policy Engine	Context Required
Tool Invocation	Can agent (on behalf of user) call this tool now?	Agent + User	Cedar AVP	time, risk, geo, mfa, data_class
Memory Read	Can agent read memory scope for this user session?	Agent	Cedar AVP	session_id, memory_scope, tenant
Knowledge Access	Can agent retrieve this document?	Agent	Cedar AVP	doc_classification, user_clearance
API Call	Can agent call this downstream API endpoint?	Agent	Cedar AVP	api_scope, rate_limit, tenant
Output Return	Is the output data classification permitted?	Agent	Cedar AVP	output_class, user_clearance
Human Approval	Does this action require human approval?	Action	Cedar AVP	amount, risk_score, action_type
Sub-Agent Spawn	Can this agent create a sub-agent?	Agent	Cedar AVP	agent_tier, scope_constraints

2. Tool Authorization Architecture

Tools are the action surface of an AI agent. Each tool invocation must be independently authorized. Tool authorization must consider the tool's data classification, the invoking principal's capabilities, and the runtime context.

2.1 Tool Authorization Policy Examples

SQL Query Tool — DBA Only, No Off-Hours

```
// SQL Query Tool: restricted to DBA-capable principals // during business hours with
non-elevated risk permit( principal, action == BankAI::Action::"InvokeTool", resource ==
BankAI::Tool::"SQLQueryTool" ) when {
principal.capabilities.contains("can_query_production_db") && context.businessHours == true &&
context.riskScore < 50 && principal.mfaVerified == true }; // Additional restriction: no bulk
exports without senior approval forbid( principal, action == BankAI::Action::"InvokeTool",
resource == BankAI::Tool::"SQLQueryTool" ) when { context.queryType == "BULK_EXPORT" &&
!principal.capabilities.contains("can_export_bulk_data") };
```

SAP ERP Tool — Finance Department Only

```
// SAP tool access: Finance department, own geography permit( principal, action ==
BankAI::Action::"InvokeTool", resource == BankAI::Tool::"SAPTool" ) when {
principal.capabilities.contains("can_access_sap") && principal.businessUnit == "FINANCE" &&
resource.allowedGeographies.contains(principal.geography) && context.sessionAge < 3600 //
session must be fresh (< 1 hour) };
```

Delete Tool — Strict Business Hours + Approval

```
// Destructive operations require approval AND business hours forbid( principal, action ==
BankAI::Action::"InvokeTool", resource == BankAI::Tool::"DeleteRecordTool" ) unless {
context.humanApprovalStatus == "APPROVED" && context.businessHours == true && context.riskScore
< 30 && principal.capabilities.contains("can_delete_records") };
```

External SaaS API Tool — Tenant Isolation

```
// Agents can only call SaaS tools for their own tenant permit( principal is BankAI::Agent,
action == BankAI::Action::"InvokeTool", resource is BankAI::Tool ) when { resource has tenantId
&& principal.tenantId == resource.tenantId && principal.delegatedFrom.capabilities.contains(
resource.requiredCapability ) };
```

2.2 Tool Capability Taxonomy

Tool Category	Example Tools	Required Capability	Additional Controls
Data Query	SQL Tool, DynamoDB Tool, Athena Tool	can_query_database	Business hours, MFA, risk < 50
Financial Operations	Payment Tool, Transfer Tool, FX Tool	can_approve_payment	Dual approval for > \$10K, MFA, low risk
ERP/CRM Access	SAP Tool, Salesforce Tool	can_access_erp	Department match, geography, session age
Document Operations	SharePoint Tool, S3 Tool	can_access_documents	Classification-based, DLP active

Tool Category	Example Tools	Required Capability	Additional Controls
Communication	Email Tool, Teams Tool, Slack Tool	can_send_communication	Recipient validation, DLP scan
Code Execution	Lambda Tool, CodeBuild Tool	can_execute_code	Sandbox only, output size limit
Destructive Operations	Delete Tool, Purge Tool	can_delete_records	Business hours, approval, risk < 30
External APIs	Vendor API Tool, Partner Tool	can_call_external_api	Tenant isolation, rate limit
HR Systems	Workday Tool, HR Portal Tool	can_view_hr_records	Own geography only, MFA

3. Context-Aware Authorization

Static role-based authorization is insufficient for Agentic AI. Authorization decisions must be context-aware — incorporating runtime signals that affect the risk and appropriateness of an action at the moment it is requested.

3.1 Contextual Signals Catalog

Signal	Source	Used For	Example Policy Impact
Timestamp / Business Hours	System clock	Restrict destructive actions to office hours	Delete operations forbidden 18:00–08:00
Risk Score (0–100)	Risk Engine (AWS Fraud Detector)	Elevate controls for high-risk sessions	Risk > 70: require additional MFA step
Device Compliance	Microsoft Endpoint Manager / Intune	Require managed device for sensitive data	PII access denied on unmanaged device
Geographic Location	IP geolocation / VPN context	Enforce data residency, geo-restrictions	EU data cannot be accessed from US
MFA Method & Age	Entra ID AMR claim	Require strong auth for high-value actions	Payments: phishing-resistant MFA only
Session Age (minutes)	Token issuance time	Require re-authentication for aged sessions	Sessions > 60 min: step-up auth for admin
Network Zone	VPC subnet tags, Zero Trust NAC	Differentiate corporate vs remote access	CORPORATE allows all; REMOTE restricted
Agent Confidence Score	LLM reasoning chain	Limit tool access if agent is uncertain	Confidence < 80%: require human review
Prompt Classification	Content classifier (Bedrock Guardrails)	Block prompt injection, jailbreak attempts	Injected prompt: deny and alert
Data Sensitivity	AWS Macie, data catalog	Enforce proportional access controls	TOP_SECRET: additional approval required
Purpose / Task Type	Agent task metadata	Restrict tools to task-appropriate subset	Tax task: only tax tools accessible
Approval Status	Workflow engine (Step Functions)	Gate on explicit prior approval	Payment > \$50K: APPROVED status required
Threat Score	AWS GuardDuty, SIEM	Emergency circuit breaker	Active threat: deny all non-essential access

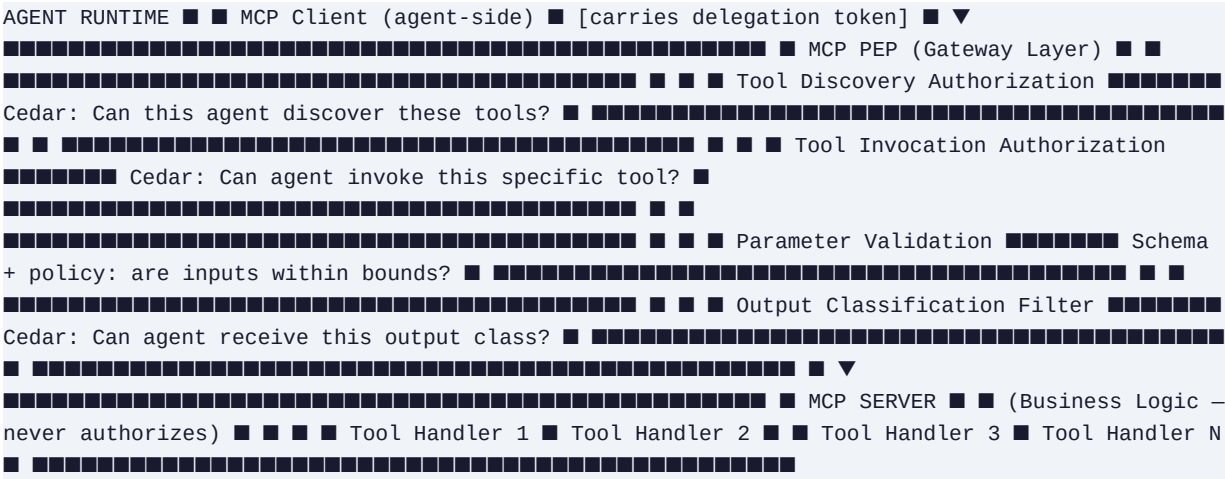
3.2 Context Object Structure for Cedar Evaluation

```
{ "requestId": "req-7f3a9b2e", "timestamp": "2025-06-26T09:47:00Z", "businessHours": true,
  "riskScore": 22, "deviceCompliant": true, "networkZone": "CORPORATE", "geography": "GB",
  "mfaMethod": "FIDO2", "mfaAge": 8, "sessionAge": 14, "agentConfidenceScore": 93,
  "promptClassification": "BENIGN", "dataClassification": "CONFIDENTIAL", "purposeCode":
  "PAYMENT_PROCESSING", "humanApprovalStatus": "NOT_REQUIRED", "threatScore": 5, "dlpActive":
  true, "tenantId": "bank-prod", "agentId": "agent-bedrock-payments-01", "workflowId":
  "wf-payment-4421" }
```

4. MCP Server Security Architecture

The Model Context Protocol (MCP) is the emerging standard for connecting AI agents to tools, APIs, and data sources. Every enterprise MCP deployment must treat the MCP boundary as a security perimeter requiring full policy enforcement.

4.1 MCP Authorization Architecture



4.2 MCP Security Requirements

Security Requirement	Implementation	Policy Engine
Tool Discovery Authorization	Cedar policy: agent type X may only discover tools in permitted tool group G	Cedar AVP
Tool Invocation Authorization	Every tool call passes through PEP; Cedar evaluates per-tool policies	Cedar AVP
Dynamic Tool Registration	New tools cannot be registered without policy store update and CI/CD approval	Cedar PAP
MCP Authentication	MCP client must present valid delegation token (RFC 8693)	JWT validation layer
Parameter Input Validation	Tool parameters validated against JSON Schema AND policy constraints	Cedar + schema
Output Classification	MCP server tags outputs with data classification; PEP filters based on agent clearance	Cedar AVP
Remote MCP TLS	Remote MCP connections require mutual TLS; certificate pinning for production	AWS Certificate Manager
Local MCP Sandbox	Local MCP processes run in isolated containers with restricted IAM roles	OPA + IAM
Tool Rate Limiting	Per-agent, per-tool rate limits enforced at MCP PEP	PEP middleware
Audit Logging	Every MCP tool invocation logged with principal, tool, parameters hash, outcome	CloudTrail

4.3 MCP Tool Cedar Policies

```
// MCP Tool Discovery – restrict by agent type permit( principal is BankAI::Agent, action ==
BankAI::Action::"DiscoverTools", resource is BankAI::ToolGroup ) when { resource ==
BankAI::ToolGroup::"PaymentTools" && principal.agentType == "PaymentAgent" &&
principal.delegatedFrom.capabilities.contains("can_approve_payment") }; // MCP Tool Invocation
– full context evaluation permit( principal is BankAI::Agent, action ==
BankAI::Action::"InvokeMCPTool", resource is BankAI::Tool ) when {
principal.delegatedFrom.capabilities.contains(resource.requiredCapability) && context.riskScore
< 60 && context.tenantId == resource.tenantId && context.promptClassification != "INJECTION" };
// MCP Anti-injection: block all tool calls if prompt injection detected forbid( principal,
action == BankAI::Action::"InvokeMCPTool", resource ) when { context.promptClassification ==
"INJECTION" };
```

5. Multi-Agent Workflow Authorization

Multi-agent architectures introduce the confused deputy problem: a downstream agent may be granted permissions by an orchestrating agent that the orchestrator does not itself possess. Cedar policies must explicitly address agent-to-agent delegation boundaries.

5.1 Multi-Agent Trust Architecture

```

USER (with JWT) ■ ▼ ORCHESTRATOR AGENT (Level 0) • Has delegated scope from user • Scope is
constrained by user capabilities • Cannot grant sub-agents MORE than its own scope ■ ■■■■
SPECIALIST AGENT A (Level 1 – Payment Processing) ■ • Scope = INTERSECT(Orchestrator scope,
Agent A permitted tools) ■ • Cedar evaluates BOTH the delegation AND the tool permission ■
■■■■ SPECIALIST AGENT B (Level 1 – Data Retrieval) ■ • Scope = INTERSECT(Orchestrator scope,
Agent B permitted tools) ■ • No cross-contamination between Agent A and Agent B scopes ■ ■■■■
HUMAN APPROVAL GATE • Triggered by Cedar obligation on high-risk actions • AWS Step Functions
Human Task • Approval stored in context, verified by Cedar at next step AUTHORIZATION PRINCIPLE:
Sub-agent scope ⊆ Orchestrator scope ⊆ User delegated scope ⊆ User capabilities

```

5.2 Human-in-the-Loop Authorization Pattern

Cedar can return obligations alongside Allow/Deny decisions. An obligation instructs the PEP to take a specific action before execution proceeds — the most important being to require human approval:

```

// Payment approval obligation: trigger human review for large amounts permit( principal is
BankAI::Agent, action == BankAI::Action::"InvokeTool", resource ==
BankAI::Tool::"PaymentExecutionTool" ) when {
principal.delegatedFrom.capabilities.contains("can_approve_payment") && context.paymentAmount
<= 10000 }; // Payments above threshold require human approval obligation permit( principal is
BankAI::Agent, action == BankAI::Action::"InvokeTool", resource ==
BankAI::Tool::"PaymentExecutionTool" ) when {
principal.delegatedFrom.capabilities.contains("can_approve_payment") && context.paymentAmount >
10000 && context.humanApprovalStatus == "APPROVED" && context.approvalTimestamp >
(context.currentTime - 300) // within 5 minutes };

```

■ NOTE

Cedar Obligation Pattern: When Cedar returns ALLOW with an obligation 'REQUIRE_HUMAN_APPROVAL', the PEP must NOT execute the action immediately. Instead it must trigger the approval workflow (AWS Step Functions Human Task), await the approval, and re-evaluate the Cedar policy with humanApprovalStatus='APPROVED' in the context. The PEP is responsible for enforcing obligations.

6. End-to-End Sequence Diagrams

6.1 REST API Agent Authorization Flow

```
User API GW Lambda Auth Claims Svc Cedar AVP Agent Runtime Tool ■■■■ POST
/■■■■■ (JWT) ■■■■ authorizer ■■■■ validate ■■■■
■■■■ JWT sig ■■■■ claims ■■■■ normalize ■■■■
canonical ■■■■ IsAuthorized ■■■■ ALLOW ■■■■
200 policy ■■■■ route ■■■■
tool auth ■■■■ ALLOW ■■■■ invoke ■■■■ result
■■■■ log ■■■■ response ■■■■
```

6.2 Multi-Agent Workflow Authorization Sequence

[illegible]